

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5

8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I Kadhirezhilan. D [192511228] certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Online Shopping Platform – Normalize to 3NF

Given Relation:

Order (OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Step 1: Identify Functional Dependencies

The important functional dependencies are:

1. **OrderID** → **CustID**
2. **CustID** → **CustName**
3. **ProdID** → **ProdName, Price**
4. **(OrderID, ProdID)** → **Qty**

Therefore:

- **CustID** determines the customer name.
- **ProdID** determines product name and price.
- The quantity depends on the particular product within an order.

The candidate key of the original relation is:

(OrderID, ProdID)

Step 2: Identify Partial and Transitive Dependencies

There are partial dependencies:

- **OrderID** → **CustID**
- **ProdID** → **ProdName, Price**

because these attributes depend only on part of the composite key.

There is also a transitive dependency:

OrderID → **CustID** → **CustName**

Step 3: Decompose into 3NF

CUSTOMER

Customer (CustID, CustName)

Primary Key: **CustID**

FD:

CustID → **CustName**

ORDER

Orders(OrderID, CustID)

Primary Key: **OrderID**

Foreign Key: **CustID** → **Customer(CustID)**

PRODUCT

Product(ProdID, ProdName, Price)

Primary Key: **ProdID**

FD:

ProdID → **ProdName, Price**

ORDER_ITEM

OrderItem(OrderID, ProdID, Qty)

Primary Key: **(OrderID, ProdID)**

Foreign Keys:

- **OrderID** → **Orders(OrderID)**
- **ProdID** → **Product(ProdID)**

Final 3NF Design

Customer (CustID, CustName)

Orders (OrderID, CustID)

Product (ProdID, ProdName, Price)

OrderItem (OrderID, ProdID, Qty)

Justification

The decomposition removes partial and transitive dependencies and therefore eliminates unnecessary redundancy. Customer information is stored only once, product information is maintained independently, and order-specific quantities are stored in OrderItem. The decomposition preserves the important functional dependencies and provides a lossless relational design.

Q2. Hospital System – Normalize to BCNF

Given Relation

Patient (PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Functional Dependencies

Assuming each patient has one assigned doctor and each doctor belongs to one specialization and ward:

1. **PID $\rightarrow$ PName, DocID, WardNo**
2. **DocID $\rightarrow$ DocName, Specialization, WardNo**
3. **WardNo $\rightarrow$ WardName**

The candidate key of the original relation is:

PID

Anomalies

1. Update Anomaly

If a doctor's specialization changes, the information must be updated for every patient assigned to that doctor.

2. Insertion Anomaly

A new doctor cannot be stored unless at least one patient is associated with that doctor.

3. Deletion Anomaly

If the last patient of a doctor is deleted, information about the doctor may also be lost.

BCNF Violation

For:

DocID $\rightarrow$ DocName, Specialization, WardNo

DocID is not a candidate key of the original relation because PID is the key.

Therefore, this dependency violates BCNF.

Similarly:

WardNo $\rightarrow$ WardName

violates BCNF because WardNo is not a candidate key of the original relation.

BCNF Decomposition

PATIENT

Patient (PID, PName, DocID)

Primary Key: **PID**

DOCTOR

Doctor (DocID, DocName, Specialization, WardNo)

Primary Key: **DocID**

WARD

Ward(WardNo, WardName)

Primary Key: **WardNo**

Relationships

- Patient.DocID $\rightarrow$ Doctor.DocID
- Doctor.WardNo $\rightarrow$ Ward.WardNo

Justification

Each determinant in the decomposed relations is a candidate key. Hence, the resulting relations satisfy BCNF. The design removes update, insertion, and deletion anomalies while improving data integrity and reducing redundancy.

Q3. University Enrollment – 2NF Decomposition

Given Relation

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

Functional Dependencies

1. **SID $\rightarrow$ SName**
2. **CourseID $\rightarrow$ CourseName, Faculty**
3. **(SID, CourseID) $\rightarrow$ Grade**

The candidate key is:

(SID, CourseID)

Partial Dependencies

Since:

SID $\rightarrow$ SName

SName depends only on SID.

Also:

CourseID $\rightarrow$ CourseName, Faculty

CourseName and Faculty depend only on CourseID.

These are partial dependencies because the key is composite.

2NF Decomposition

STUDENT

Student (SID, SName)

Primary Key: **SID**

COURSE

Course (CourseID, CourseName, Faculty)

Primary Key: **CourseID**

ENROLLMENT

Enrollment(SID, CourseID, Grade)

Primary Key: **(SID, CourseID)**

Foreign Keys:

- **SID $\rightarrow$ Student(SID)**
- **CourseID $\rightarrow$ Course(CourseID)**

Justification

The decomposition removes all partial dependencies. Student details depend completely on SID, course details depend completely on CourseID, and Grade depends on the complete combination of SID and CourseID.

Therefore, the resulting design satisfies **2NF** and avoids unnecessary duplication of student and course information.

Q4. Airline Booking – Normalize to BCNF

Given Relation

Booking (FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Assumptions

Assume:

- A passenger is uniquely identified by PassID.
- Each flight is identified by FlightNo.
- A flight operates between one departure and arrival city on a particular date.
- A seat is assigned to a passenger for a particular flight/date.

Functional Dependencies

1. **PassID $\rightarrow$ PassName**
2. **FlightNo $\rightarrow$ DeptCity, ArrCity**
3. **(FlightNo, Date, PassID) $\rightarrow$ Seat**

A practical candidate key is:

(FlightNo, Date, PassID)

BCNF Decomposition

PASSENGER

Passenger (PassID, PassName)

Primary Key: **PassID**

FD:

PassID $\rightarrow$ PassName

FLIGHT

Flight (FlightNo, DeptCity, ArrCity)

Primary Key: **FlightNo**

FD:

FlightNo $\rightarrow$ DeptCity, ArrCity

BOOKING

Booking (FlightNo, Date, PassID, Seat)

Primary Key:

(FlightNo, Date, PassID)

Foreign Keys:

- FlightNo → Flight (FlightNo)
- PassID → Passenger (PassID)

Additional Integrity Constraint

A seat should normally be unique for a particular flight and date:

(FlightNo, Date, Seat) must be UNIQUE.

Justification

Passenger details and flight-route details are separated from the booking transaction. Each determinant in the resulting relations is a candidate key, satisfying BCNF under the stated assumptions. The design also prevents duplicate seat allocation when the appropriate UNIQUE constraint is applied.

Q5. HR System – Functional and Transitive Dependencies to 3NF

Assumed HR Relation

For this case, assume the multinational HR system stores:

**Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName,
CountryID, CountryName, Salary)**

Functional Dependencies

1. **EmpID → EmpName, DeptID, ManagerID, CountryID, Salary**
2. **DeptID → DeptName, ManagerID**
3. **ManagerID → ManagerName**
4. **CountryID → CountryName**

Transitive Dependencies

The following transitive dependencies exist:

EmpID → DeptID → DeptName

Therefore:

EmpID → DeptName

Also:

EmpID → ManagerID → ManagerName

Therefore:

EmpID → ManagerName

And:

EmpID → CountryID → CountryName

Therefore:

EmpID → CountryName

These dependencies cause redundancy.

3NF Decomposition

EMPLOYEE

Employee (EmpID, EmpName, DeptID, ManagerID, CountryID, Salary)

Primary Key: **EmpID**

DEPARTMENT

Department (DeptID, DeptName, ManagerID)

Primary Key: **DeptID**

MANAGER

Manager (ManagerID, ManagerName)

Primary Key: **ManagerID**

COUNTRY

Country (CountryID, CountryName)

Primary Key: **CountryID**

Justification

The decomposition removes transitive dependencies by storing department, manager, and country information separately. Non-key attributes now depend on the key, the whole key, and nothing but the key.

This reduces redundancy and prevents update, insertion, and deletion anomalies while maintaining referential integrity through foreign keys.

Note: The exact FDs depend on the HR business rules; the above is the normalized design under the stated assumptions.

Q6. Library System – MVDs and 4NF

Given Relation

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Functional Dependencies

1. **MemberID** $\rightarrow$ **MemberName**
2. **BookID** $\rightarrow$ **Title, Author, Genre**

The original relation contains information about members, books, and borrowing transactions.

Multivalued Dependency

If a member can borrow multiple books independently of other member-related values, we can represent:

MemberID $\twoheadrightarrow$ **BookID**

However, an important point is that **BorrowDate belongs to the specific Member–Book borrowing event**. Therefore, treating MemberID $\twoheadrightarrow$ BorrowDate as an independent MVD would lose the association between a book and its borrowing date.

4NF Design

MEMBER

Member(MemberID, MemberName)

Primary Key: **MemberID**

BOOK

Book(BookID, Title, Author, Genre)

Primary Key: **BookID**

BORROW

Borrow(MemberID, BookID, BorrowDate)

Primary Key:

(MemberID, BookID, BorrowDate)

Foreign Keys:

- $\text{MemberID} \rightarrow \text{Member}(\text{MemberID})$
- $\text{BookID} \rightarrow \text{Book}(\text{BookID})$

Why this is 4NF

The independent entity information is separated from the borrowing relationship. Book details are not repeated for every borrowing transaction, and member details are not repeated for every borrowed book.

The Borrow relation preserves the important association:

Member + Book $\rightarrow$ BorrowDate

Thus, the decomposition avoids the redundancy caused by independent multivalued facts while preserving the borrowing event.

Important Observation

A genuine MVD should only be introduced when the business semantics establish independent multiple values. Since the supplied case does not explicitly state that borrowing dates are independent of books, the above design is the safer lossless representation.

Q7. Telecom Billing – Candidate Keys, Primary Key and FDs

Assumed Telecom Billing Schema

Since the question does not provide an explicit relation, assume:

Billing(BillID, CustomerID, PhoneNo, CustomerName, PlanID, PlanName, CallID, CallDuration, Rate, Amount)

Functional Dependencies

1. **BillID $\rightarrow$ CustomerID, PhoneNo, PlanID**
2. **CustomerID $\rightarrow$ CustomerName**
3. **PlanID $\rightarrow$ PlanName, Rate**
4. **CallID $\rightarrow$ PhoneNo, CallDuration**
5. **(CallID, Rate) $\rightarrow$ Amount**

Candidate Keys

Under the assumption that every bill has a unique BillID:

Candidate Key = BillID

If CallID uniquely identifies each call:

Candidate Key = CallID

However, these are keys for different conceptual entities and should not be mixed in one universal relation.

Recommended Primary Keys

- **Bill (BillID, CustomerID, PhoneNo, PlanID)**
- **Customer (CustomerID, CustomerName)**
- **Plan (PlanID, PlanName, Rate)**
- **Call (CallID, PhoneNo, CallDuration, Amount)**

Foreign Keys

- $\text{Bill.CustomerID} \rightarrow \text{Customer.CustomerID}$
- $\text{Bill.PlanID} \rightarrow \text{Plan.PlanID}$
- $\text{Call.PhoneNo} \rightarrow \text{Customer/Subscriber.PhoneNo}$

Analysis

The main determinants are:

BillID $\rightarrow$ bill information

CustomerID $\rightarrow$ CustomerName

PlanID $\rightarrow$ PlanName, Rate

These dependencies show that storing everything in a single billing table would produce repeated customer and plan information.

Conclusion

Candidate keys must be identified from the business rules before normalization. The primary key should be selected from the candidate keys, preferably using the stable unique identifier **BillID** for the bill entity.

Note: Because the assessment question does not specify the complete telecom schema or business rules, the keys and FDs above are explicitly based on the stated assumptions.

Q8. Retail Inventory – Complete Normalization

Given Relation

Product (PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Functional Dependencies

1. **PID** $\rightarrow$ PName, SupplierID, Category, Price, Warehouse
2. **SupplierID** $\rightarrow$ SupplierName

Therefore:

PID $\rightarrow$ **SupplierID** $\rightarrow$ SupplierName

This is a transitive dependency.

1NF

The relation is assumed to contain atomic values.

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

2NF

Assuming PID is the single-attribute key, there can be no partial dependency on a composite key.

Therefore, it is already in **2NF**.

3NF

There is a transitive dependency:

PID → SupplierID → SupplierName

Therefore, decompose.

Final Normalized Design

PRODUCT

Product (PID, PName, SupplierID, Category, Price, Warehouse)

Primary Key: **PID**

SUPPLIER

Supplier (SupplierID, SupplierName)

Primary Key: **SupplierID**

BCNF Check

In Product:

PID → PName, SupplierID, Category, Price, Warehouse

PID is the candidate key.

In Supplier:

SupplierID → SupplierName

SupplierID is the candidate key.

Therefore, both relations satisfy BCNF under the assumed business rules.

Justification

The decomposition eliminates the transitive dependency and prevents repeated supplier names. If a supplier changes its name, only one row needs to be updated.

Thus, the design progresses: $1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF$ and provides better consistency and data integrity.

Q9. School ERP – Fully Normalized Design up to BCNF

Assumed Poorly Normalized Schema

Since the question does not provide the actual School ERP relation, assume:

SchoolERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

Functional Dependencies

1. **StudentID $\rightarrow$ StudentName, ClassID**
2. **ClassID $\rightarrow$ ClassName, TeacherID**
3. **TeacherID $\rightarrow$ TeacherName**
4. **SubjectID $\rightarrow$ SubjectName**
5. **(StudentID, SubjectID) $\rightarrow$ Grade**

The composite candidate key for enrollment is: **(StudentID, SubjectID)**

Problems

There are partial dependencies: **StudentID $\rightarrow$ StudentName, ClassID** and:

SubjectID $\rightarrow$ SubjectName

There are transitive dependencies:

StudentID $\rightarrow$ ClassID $\rightarrow$ ClassName and:

StudentID $\rightarrow$ ClassID $\rightarrow$ TeacherID $\rightarrow$ TeacherName

BCNF Decomposition

STUDENT

Student(StudentID, StudentName, ClassID)

Primary Key: **StudentID**

CLASS

Class (ClassID, ClassName, TeacherID)

Primary Key: **ClassID**

TEACHER

Teacher (TeacherID, TeacherName)

Primary Key: **TeacherID**

SUBJECT

Subject (SubjectID, SubjectName)

Primary Key: **SubjectID**

ENROLLMENT

Enrollment (StudentID, SubjectID, Grade)

Primary Key:

(StudentID, SubjectID)

Foreign Keys

- Student.ClassID → Class.ClassID
- Class.TeacherID → Teacher.TeacherID
- Enrollment.StudentID → Student.StudentID
- Enrollment.SubjectID → Subject.SubjectID

BCNF Verification

For Student:

StudentID → StudentName, ClassID

StudentID is the candidate key.

For Class:

ClassID → ClassName, TeacherID

ClassID is the candidate key.

For Teacher:

TeacherID $\rightarrow$ TeacherName

TeacherID is the candidate key.

For Subject:

SubjectID $\rightarrow$ SubjectName

SubjectID is the candidate key.

For Enrollment:

(StudentID, SubjectID) $\rightarrow$ Grade

The complete composite key determines Grade.

Therefore, all determinants are candidate keys in their respective relations, satisfying BCNF.

Justification

The decomposition removes partial and transitive dependencies, minimizes redundancy, and preserves referential integrity. It also allows independent modification of student, class, teacher, and subject information without creating anomalies.

Q10. Movie Streaming Platform – Join Dependencies and 5NF

Assumed Case

The question does not provide the exact movie-streaming relation. To analyze 5NF, assume the platform maintains:

MovieStreaming(MovieID, ActorID, PlatformID)

Suppose the business rule states that a movie is associated with an actor and a streaming platform independently, and the valid combinations can be reconstructed from the following relationships:

- Movie–Actor
- Movie–Platform
- Actor–Platform

Join Dependency

The relation can have the join dependency:

(MovieID, ActorID), (MovieID, PlatformID), (ActorID, PlatformID)

This means the original relation can be reconstructed by joining these three projections.

Decomposition

MOVIE_ACTOR

MovieActor(MovieID, ActorID)

Primary Key:

(MovieID, ActorID)

MOVIE_PLATFORM

MoviePlatform(MovieID, PlatformID)

Primary Key:

(MovieID, PlatformID)

ACTOR_PLATFORM

ActorPlatform(ActorID, PlatformID)

Primary Key:

(ActorID, PlatformID)

5NF Condition

A relation is in **5NF (Project-Join Normal Form)** when every non-trivial join dependency is implied by candidate keys.

The original ternary relation contains a non-trivial join dependency. Decomposing it into the three binary relations eliminates that non-trivial join dependency from the individual relations.

Lossless-Join Verification

The original relation can be reconstructed as:

MovieActor ⋈ MoviePlatform ⋈ ActorPlatform

The common attributes between the relations allow the valid combinations to be joined.

Therefore, under the stated join-dependency assumption:

$$R = \pi(M,A)(R) \bowtie \pi(M,P)(R) \bowtie \pi(A,P)(R)$$

where:

- M = MovieID
- A = ActorID
- P = PlatformID

Thus, the decomposition is **lossless with respect to the assumed join dependency**.

Important Note

5NF cannot be determined from attribute names alone. A genuine join dependency must be specified by the application's business rules. Since the assessment does not provide those rules, this 5NF solution is based on the explicit assumption stated above.

Overall Conclusion

Normalization systematically improves the database design by identifying functional dependencies, removing partial and transitive dependencies, and decomposing relations while maintaining data integrity.

The progression can be summarized as:

$$1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF \rightarrow 4NF \rightarrow 5NF$$

- **1NF:** Removes repeating groups and ensures atomic values.
- **2NF:** Removes partial dependencies.
- **3NF:** Removes transitive dependencies.
- **BCNF:** Every determinant must be a candidate key.
- **4NF:** Removes problematic non-trivial multivalued dependencies.
- **5NF:** Removes non-trivial join dependencies.

The final normalized designs reduce redundancy, prevent insertion/update/deletion anomalies, improve consistency, and provide stronger integrity through primary and foreign-key constraints.